Secure Software Design and Development (Part II: Seven Touchpoints for Software Security)

CHAPTER 5. INTRODUCTION TO SOFTWARE SECURITY TOUCHPOINTS

BOOK: SOFTWARE SECURITY: BUILDING SECURITY IN, 1ST EDITION BY GARY MCGRAW

Seven Touchpoints

Seven Touchpoints [1/2]

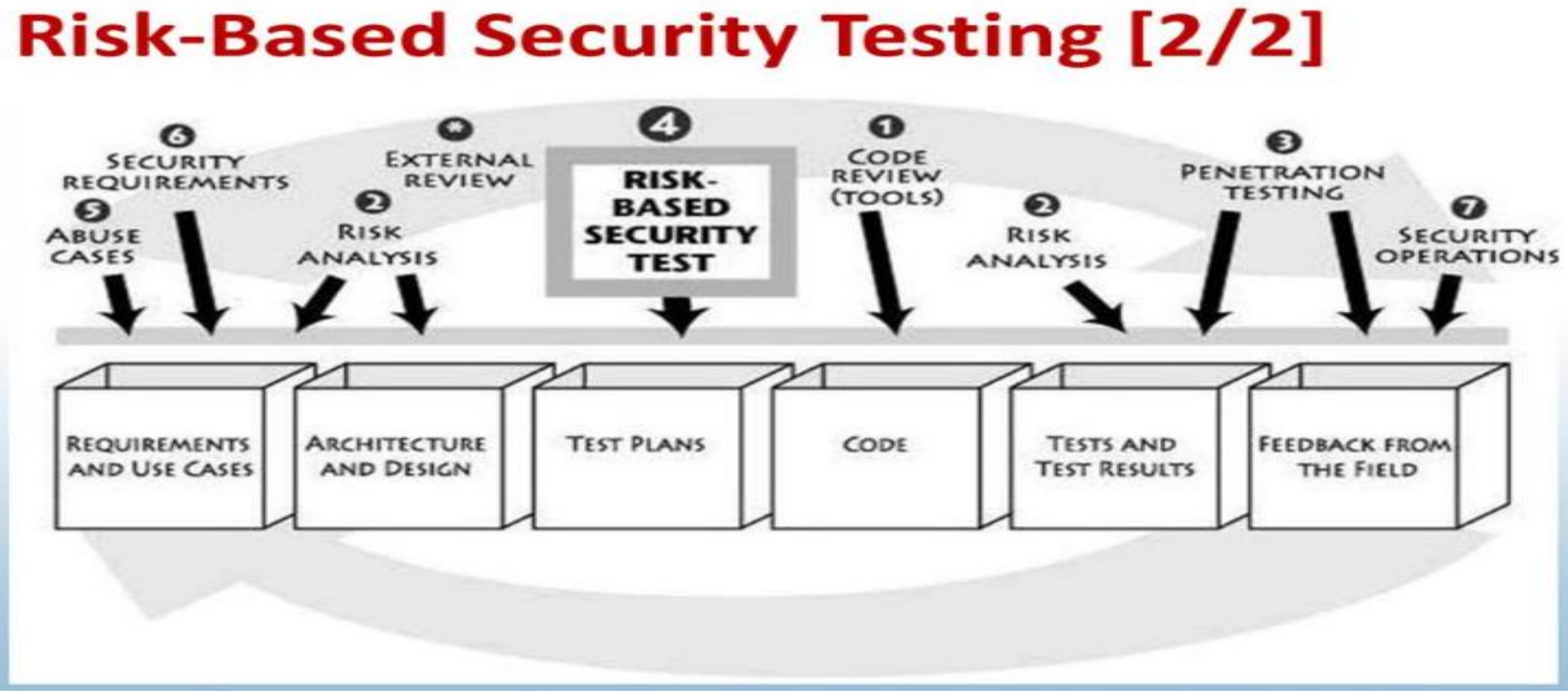
1. Code Review (Tools)
2. Architectural Risk Analysis
3. Penetration Testing
4. Risk-Based Security Testing
5. Abuse Cases
6. Security Requirements
7. Security Operations

Risk Based Security Testing

Risk-Based Security Testing [1/2]

- Risk-based testing (RBT) is a type of software testing that functions as an organizational principle used to prioritize the tests of features and functions in software, based on the risk of failure, the function of their importance and likelihood or impact of failure

Risk Based Security Testing



Risk Based Security Testing

RBT vs Penetration Testing

- RBT has similarities with penetration testing
- Differences in level of approach and timing
  - Penetration testing
    - Occurs when software is complete and installed in production environment
    - Focus on outside → in
  - RBT
    - Applied before software is complete at unit level
    - Testing environment with stubs (pre integration)

Risk Management and Security Testing

Risk Management and Security Testing

- Two diverse approaches
  - Functional Security Testing – testing security mechanism to ensure that their functionality is properly implemented
  - Adversarial Security Testing – understanding and simulating the attackers approach

Risk Based Security Testing

Risk Management and Security Testing

- Two diverse approaches
  - Functional Security Testing – testing security mechanism to ensure that their functionality is properly implemented
  - Adversarial Security Testing – understanding and simulating the attackers approach

Security Testing

How to Approach Security Testing [1/2]

- Who should perform testing?
- Functional security testing
  - Standard testing organizations using traditional approach
    - e.g. access control
- Adversarial security testing
  - Highly experienced security professional

Security Testing

How to Approach Security Testing [2/2]

- How should testing be performed?
- Make use of good tools
  - Static analyzer
- Drawbacks:
  - False positives
  - QA organizations spend little time probing
    - Time and budget constraints

Security Testing

Security Testing: Malicious Input

- Biggest problem is that software takes input
  - Register settings
  - Environment variables
  - Network configuration
- Attackers toolkits focus on input using
  - Fault injection tools
  - Grammar generators
  - Re-players

Security Testing

Security Testing: Malicious Input

- Solutions to handing malicious input
- Use white-list approach
  - Draconian white list approach
- Avoid black-list approach will not work

Risk- Based Security

Risk-Based Security Summary

- Security testing
  - Should start at feature or component/unit level testing
  - Must involve two diverse approaches
    - Functional security testing
      - Testing security mechanisms to ensure that their functionality is properly implemented
    - Adversarial security testing
      - Performing risk-based security testing motivated by understanding and simulating the attacker's approach

Abuse Cases

Abuse Cases Introduction [1/3]

- Abuse case defines an interaction between an actor and the system that results in harm to a resource associated with one of the actors, one of the stakeholders, or the system itself
- Also called misuse cases

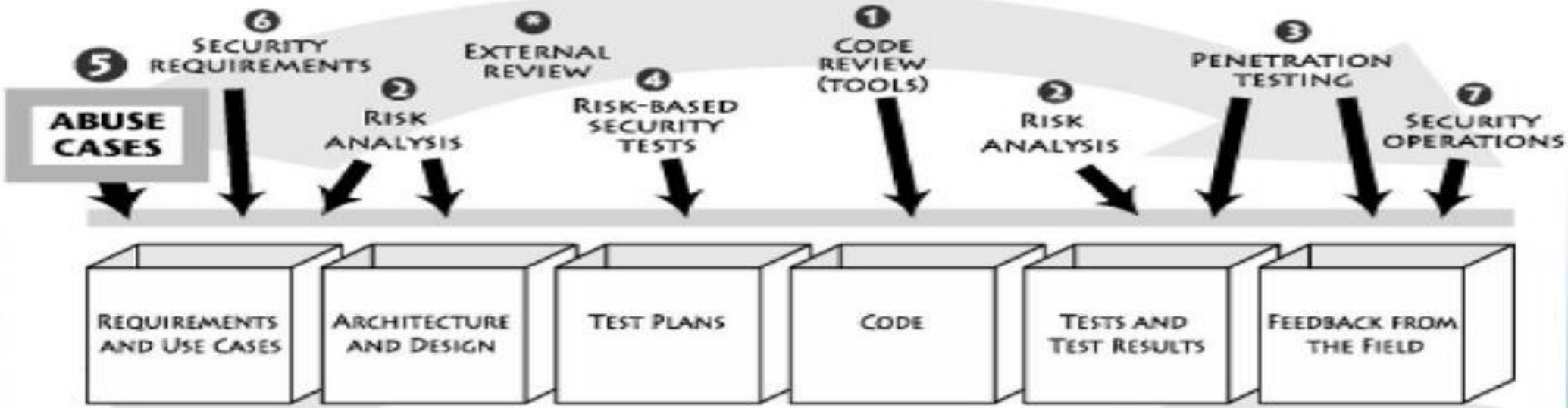
Abuse Cases

Abuse Cases Introduction [2/3]

- Think like an attacker
- What motivates an attacker?
  - Stealing money
  - Obtaining greater access level
  - Create general havoc
  - Spy on spouse
- How can the attacker accomplish its goal?

Abuse Cases

Abuse Cases Introduction [3/3]



Abuse Cases

Creating Useful Abuse Cases

- Brainstorming is the most practical method of creating abuse case
  - Combine security and reliability experts with system designers (relies heavily on experience)
  - Experts ask questions to identify possible weakness
- Often, formal methods are unnecessary in the real world

Abuse Cases

Touchpoint Process: Abuse Case Development [1/3]

- Start with requirements, standards uses cases, and list of attack patterns
- Identify your threat or actor
- Knowing the enemy that is likely to attack is a prerequisite
- Use information gathered to create abuse case

Abuse Cases

Touchpoint Process: Abuse Case Development [2/3]

- Creating anti-requirements
  - Important to think about
    - Things that you don't want your software to do
    - Requires: security analysis + requirement analysis
  - Anti-requirements
    - Provide insight into how a malicious user, attacker, thrill seeker, competitor can abuse your system
    - Considered throughout the SDLC
      - indicate what happens when a required security function is not included

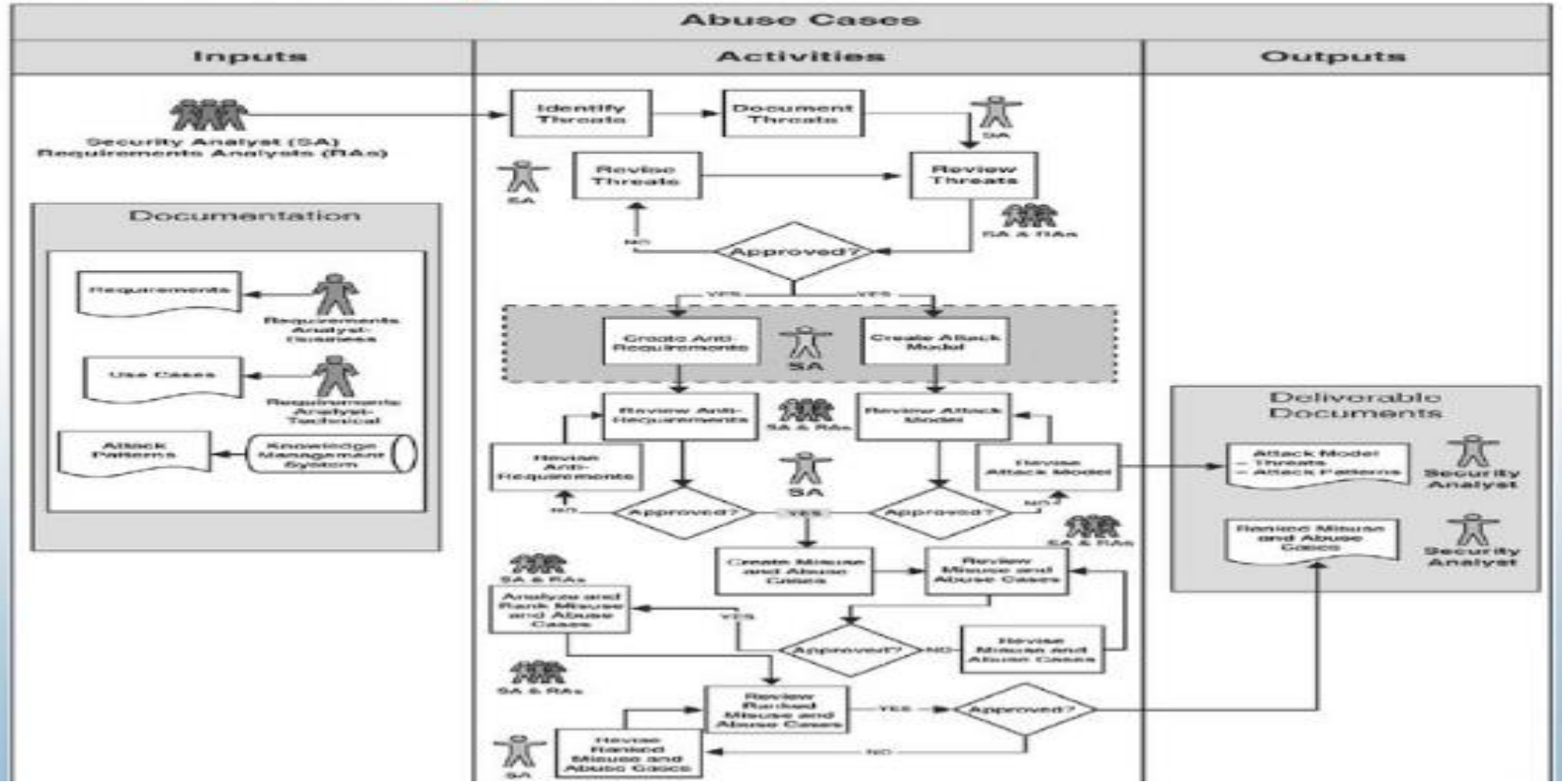
Abuse Cases

Touchpoint Process: Abuse Case Development [3/3]

- Creating attack model
  - Cycle through known attacks and determine if same attack applies to your system
  - Select those attack patterns and build cases around those patterns
  - Include anyone who can gain access to system
- Process is in Microsoft's STRIDE model

Abuse Cases

Abuse Activity Model



Abuse Cases

Abuse Case Summary

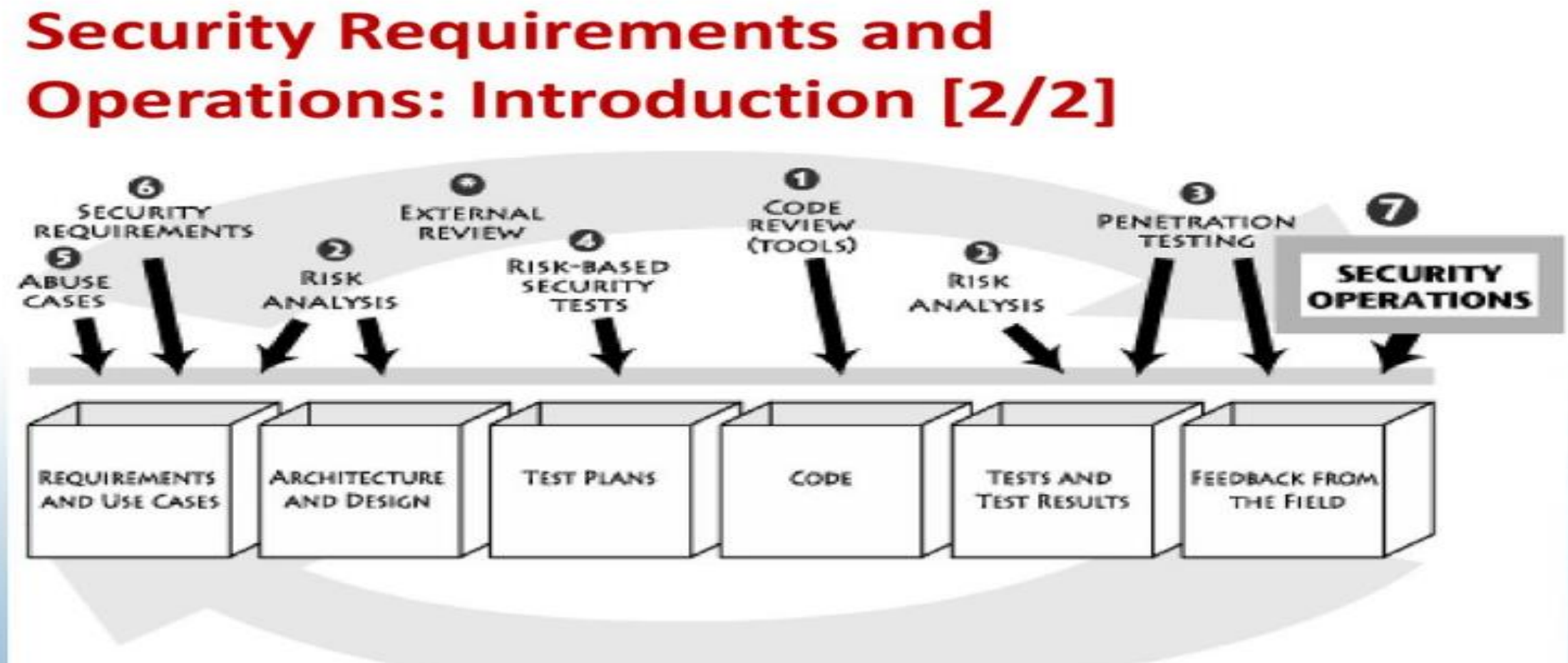
- Creating an attack model
  - Based on known attack types
  - Select attack patterns relevant to your system build abuse case around the attack patterns
  - Include anyone who can gain access to the system because threats must encompass all potential sources
  - Also need to model attacker

Security Requirements

Security Requirements and Operations: Introduction [1/2]

- Software security meets security operations
  - Traditionally, software development is far removed from information security
  - Disconnect between security and development results in software development efforts that lack any sort of contemporary understanding of technical security risks

Security Requirements



Software Security

Software Security Best Practice [1/3]

- Developers best efforts on applying touchpoints may fail
  - Lack of security domain knowledge
  - Unfamiliar with real world attacks
  - Knowledge is acquired from years of experience in the field

Software Security

Software Security Best Practice [2/3]

- Information security staff have years of responding to attacks
  - Have studied application vulnerabilities and their resulting attacks profiles in minute detail
- Few information security professionals are developers
  - Rare to find information security professionals directly involved in development projects

Software Security

Software Security Best Practice [3/3]

- Touchpoints are designed to be carried out by software security specialist in tandem with development teams
- Software security is a team effort

Software Security

Complimentary Aspects of Two Disciplines

- Requirements: Abuse cases
- Design: Business risk analysis
- Design: Architectural risk analysis
- Test Planning: Security testing
- Implementation: Code review
- System Testing: Penetration testing
- Field System: Deployment and operations

Security Requirements and Operations: Summary

- Security requirements
  - Difficult tasks
  - Should cover both overt functional security and emergent characteristics
    - Use requirements engineering approach
- Security operations
  - Integrate security operations
    - E.g., software security should be integrated with network security

Security Requirements

Security Requirements and Operations: Summary

- Security requirements
  - Difficult tasks
  - Should cover both overt functional security and emergent characteristics
    - Use requirements engineering approach
- Security operations
  - Integrate security operations
    - E.g., software security should be integrated with network security

Software Security

Software Security: Takeaways

- Software security is everyone responsibility
- Critical touchpoint to implement first
  - Code review (bugs)
  - Architectural risk analysis (flaws)
    - Account for 50% of security problem
- Utilize continuous risk management process